ESCUELA DE INGENIERÍA ELÉCTRICA

PONTIFICIA UNIVERSIDAD CATÓLICA DE VALPARAÍSO

Guía 3

Trabajo con LED y Tkinter

Curso: EIE390 - Robótica e Inteligencia Artificial
Profesor: Guillermo Cid Ampuero

Objetivos

- Objetivo 1: Comprender los fundamentos de la comunicación serial y su aplicación en la interacción entre un computador y la Raspberry Pi Pico.
- Objetivo 2: Implementar comunicación serial mediante programación en Python y MicroPython para el envío y recepción de datos.
- Objetivo 3: Desarrollar una interfaz básica utilizando Tkinter que permita enviar comandos a la Raspberry Pi Pico a través del puerto serial.

Repositorio del Curso

1 Conceptos Previos: Electrónica Básica para Actuadores

1.1 Conexión de un Diodo LED

Un Diodo Emisor de Luz (LED) es un componente polarizado, lo que significa que la corriente debe fluir en una dirección específica para que emita luz. Posee dos terminales principales:

- **Ánodo (+):** Es el terminal positivo, generalmente identificado por ser la pata más larga.
- **Cátodo (-):** Es el terminal negativo, identificado por ser la pata más corta y por tener un lado plano en el borde del encapsulado plástico del LED.

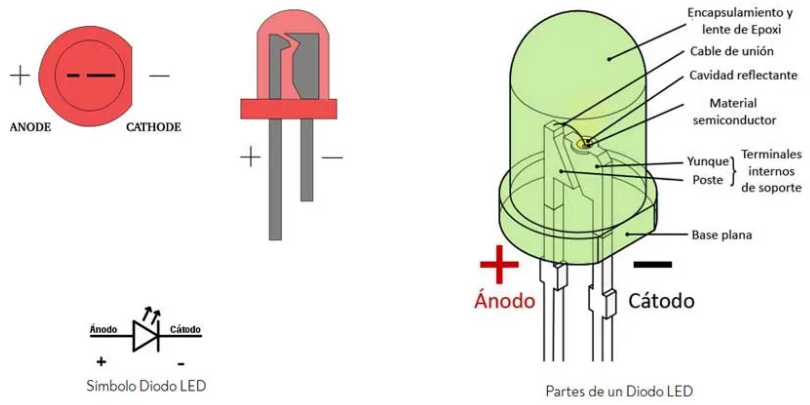

Figura 1: Esquema de conexión de un LED con su respectiva resistencia.

1.2 Conexión de un Buzzer

Un buzzer (o zumbador) es un componente electroacústico que convierte una señal eléctrica en sonido. En nuestros proyectos, se utiliza comúnmente como un actuador para emitir alertas sonoras o melodías simples. Al igual que los LEDs, la mayoría de los buzzers básicos están polarizados y cuentan con dos terminales principales que deben conectarse correctamente:

- **Terminal Positivo (+):** Suele identificarse por ser la pata más larga o por tener un signo "+" impreso en la parte superior de su carcasa plástica. Debe conectarse a un pin de propósito general (GPIO) configurado como salida en la Raspberry Pi Pico. Si se utiliza un buzzer pasivo para generar distintas frecuencias (tonos), este pin será el encargado de recibir una señal PWM (Modulación por Ancho de Pulso).
- **Terminal Negativo (- o GND):** Es el terminal restante, generalmente de pata más corta. Debe conectarse directamente a cualquiera de los pines de tierra (GND) de la placa para cerrar el circuito eléctrico.

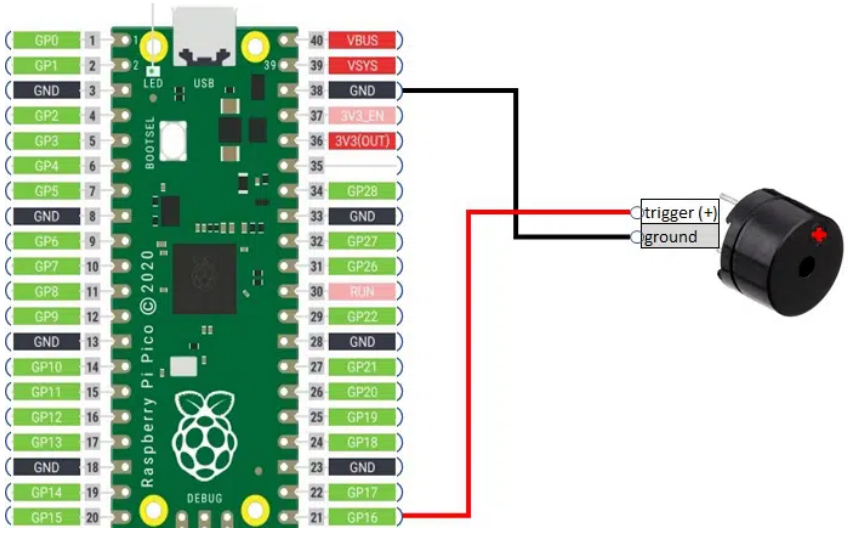

Figura 2: Esquema de conexión de un buzzer a la Raspberry Pi Pico.

Tipos de Buzzer: ¿Activo o Pasivo?

Físicamente son casi idénticos, pero su funcionamiento interno es muy distinto. Es crucial identificar cuál tiene en su kit:

- **Buzzer Activo:** Contiene su propio circuito oscilador interno. Para que emita sonido, basta con enviarle energía continua (un 1 lógico, exactamente igual que al encender un LED). Su limitación es que solo puede emitir un único tono fijo.
- **Buzzer Pasivo:** No tiene circuitería interna para generar sonido por sí solo; funciona de manera similar a un altavoz básico. Requiere que el microcontrolador le envíe una señal oscilante. Su gran ventaja es que permite generar distintas notas musicales y melodías al variar la velocidad de esa oscilación.

Concepto Clave: ¿Qué es una señal PWM?

La **Modulación por Ancho de Pulso (PWM)** es una técnica que nos permite obtener resultados analógicos utilizando pines que son estrictamente digitales (que solo pueden entregar 0V o 3,3V).
Lo logra encendiendo y apagando el pin a una velocidad tan alta que el componente conectado percibe un promedio de esa energía. Al trabajar con un buzzer pasivo usando PWM, controlamos dos parámetros fundamentales:

- **Frecuencia:** Es la cantidad de veces por segundo que la señal se enciende y apaga. Define la nota musical; una frecuencia mayor genera un sonido más agudo.
- **Ciclo de Trabajo (Duty Cycle):** Es el porcentaje de tiempo que el pin pasa encendido en comparación con el tiempo que pasa apagado durante un ciclo. En los buzzers, este parámetro nos permite controlar el **volumen** del sonido.

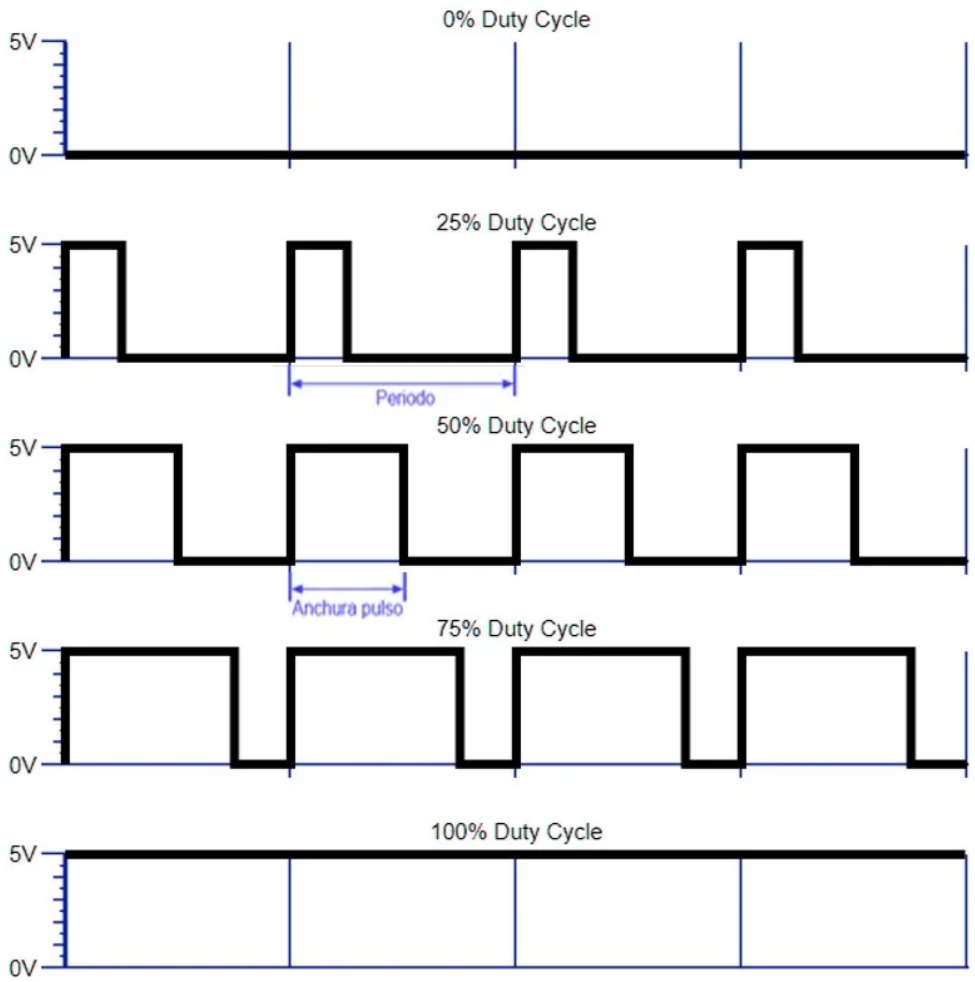

Figura 3: Señal PWM

1.3 Conexión de un Potenciómetro

Un potenciómetro es una resistencia variable mecánica que cuenta con tres terminales. Su aplicación más común al trabajar con microcontroladores es actuar como un divisor de tensión, lo que permite entregar una señal de voltaje analógico que varía según la posición de su perilla. Posee los siguientes terminales:

- **Extremos (Terminales 1 y 3):** Corresponden a los extremos fijos de la pista resistiva. Uno de ellos se conecta a la alimentación positiva y el otro a tierra. Invertir estas conexiones solo cambiará el sentido en el que aumenta o disminuye la lectura al girar la perilla.
- **Cursor o Wiper (Terminal 2):** Es el terminal central que se desliza sobre la pista resistiva. Entrega un voltaje variable entre 0V y 3.3V, y debe conectarse a un pin configurado como entrada analógica (ADC) en la Raspberry Pi Pico.

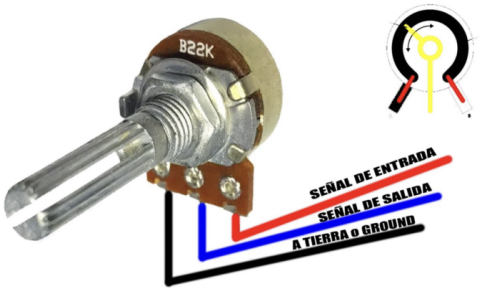

Figura 4: Esquema de conexión de un potenciómetro como divisor de tensión.

1.4 Uso Seguro y Clasificación de los Pines en la Raspberry Pi Pico

La Raspberry Pi Pico opera estrictamente con una lógica de **3.3V** (sus pines no toleran 5V). Dependiendo del componente a utilizar en este laboratorio, es fundamental seleccionar los pines adecuados y comprender sus limitaciones eléctricas:

- **Pines Digitales (para LEDs y Buzzers):** Operan con señales de encendido y apagado. Pueden utilizar casi cualquier pin de propósito general (etiquetados como GP0 a GP22, o GP26 a GP28). No obstante, la **corriente total sumada** que sale de todos los pines al mismo tiempo no debe exceder un límite de seguridad (aproximadamente $50\,mA$ en total).
- **Pines Analógicos o ADC (para el Potenciómetro):** A diferencia de los LEDs, el potenciómetro entrega un voltaje variable continuo. Para leerlo, no podemos utilizar cualquier pin, sino aquellos con Conversor Analógico a Digital (ADC). En la placa, los únicos tres pines físicos expuestos para esto son: **GP26 (Pin físico 31), GP27 (Pin físico 32) y GP28 (Pin físico 34)**. Es fundamental no confundir el número del GPIO con el número físico del pin.
- **Pines de Alimentación (VCC y GND):** Para alimentar los extremos del potenciómetro, **deben utilizar exclusivamente el pin de 3.3V** (etiquetado como 3V3(OUT), correspondiente al **Pin físico 36**) y algún pin GND de la placa (como el **Pin físico 33 o 38**). **Nunca utilicen el pin de 5V (VBUS)**. Ingresar una tensión mayor a 3.3V en una entrada analógica destruirá el conversor y posiblemente todo el microcontrolador.

Finalmente, es importante considerar que al realizar la lectura del potenciómetro mediante MicroPython, el valor analógico se escala internamente a 16 bits. Esto significa que, independientemente del hardware de 12 bits de la placa, por código obtendrán un rango de valores que irá desde 0 (0V) hasta 65535 (3.3V).

2 Actividad 1: Montaje del Hardware y Generación de Tonos

2.1 Objetivo

Familiarizarse con el uso de actuadores electroacústicos mediante la conexión de un buzzer a la Raspberry Pi Pico. El estudiante deberá configurar un pin de salida y utilizar la Modulación por Ancho de Pulso (PWM) para generar una alerta sonora, verificando su funcionamiento a través de la ejecución de un script en MicroPython.

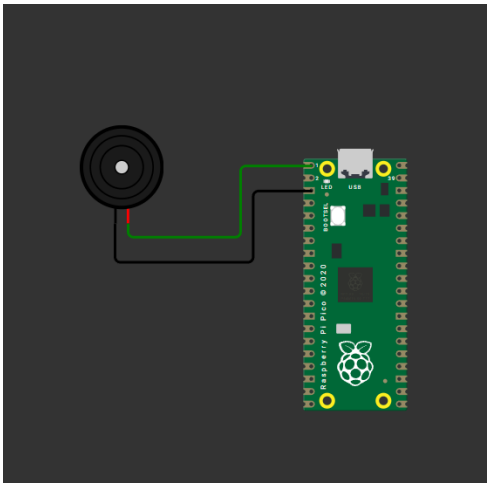

Figura 5: Esquema de conexión del buzzer a la Raspberry Pi Pico.

2.2 Materiales Requeridos

- 1x Raspberry Pi Pico
- 1x Protoboard (opcional, dependiendo de los cables disponibles)
- 1x Buzzer (pasivo o activo)
- Cables Jumper

2.3 Instrucciones

1. **Montaje del circuito:** Identifique los terminales de su buzzer. Conecte el terminal positivo (cable rojo o pata más larga) al pin **GP0** de la Raspberry Pi Pico. Conecte el terminal negativo (cable negro o pata más corta) al pin **GND** adyacente (Pin físico 3).
2. **Carga del firmware:** Abra en Thonny el archivo de código proporcionado para esta actividad.
3. **Prueba de funcionamiento:** Ejecute el script y escuche el resultado. Detenga el programa usando `Ctrl+C` en la consola.

Conceptos Clave del Código:

Para que el buzzer emita sonido, el código no envía energía continua, sino "pulsos" muy rápidos utilizando **PWM (Modulación por Ancho de Pulso)**. Las instrucciones fundamentales que hacen esto posible son:

- `PWM(Pin(0))`**:** Habilita el pin GP0 para enviar señales eléctricas oscilantes.

- `freq(1000)`**:** Define el tono o "nota"del sonido (en Hertz). Un valor mayor genera un sonido más agudo, y uno menor, más grave.
- `duty_u16(32768)`**:** Actúa como control de volumen. Al asignarle la mitad de su capacidad máxima (es decir, 32768), se logra el volumen máximo físico del componente. Asignar un valor de `0` produce silencio absoluto. **¿De dónde sale el número 65535?** La instrucción lleva el sufijo `_u16` porque la placa procesa esta señal utilizando una resolución interna de **16 bits**. En informática, esto significa que el microcontrolador puede dividir la señal en 2^{16} escalones posibles (65536 valores). Como el sistema empieza a contar desde el 0, el límite máximo permitido (el 100 %) corresponde exactamente a **65535**.

Existen dos versiones diferentes del código. La diferencia principal radica en cómo la Raspberry Pi Pico le envía la señal para generar el sonido:

- **Código para Buzzer Pasivo (Usando PWM):** Este tipo de componente funciona como un pequeño altavoz básico. No sabe qué sonido emitir por sí solo, por lo que el código utiliza **PWM (Modulación por Ancho de Pulso)** para enviarle una frecuencia específica (por ejemplo, 1000 Hz) y así obligarlo a generar un tono. El código se encuentra en **1_buzzer_pasivo.py**
- **Código para Buzzer Activo (Señal Digital):** Este componente ya tiene un circuito interno que genera su propio sonido. Solo necesita recibir energía para empezar a sonar. Por lo tanto, el código es mucho más simple y lo trata exactamente igual que a un LED: envía un **1 lógico (Encendido)** para que suene y un **0 lógico (Apagado)** para silenciarlo. El código se encuentra en **1_buzzer_activo.py**

- **Codigo 1: 1_buzzer_activo.py**
- **Codigo 1: 1_buzzer_pasivo.py**

2.4 Creación de un Musical

Se implementará un sistema de control por terminal que permite elegir entre diferentes melodías. Este código utiliza conceptos avanzados de programación:

- **Diccionarios y Tuplas:** Se utilizan para mapear las notas musicales a sus frecuencias reales en Hertz y para organizar las melodías como secuencias de (Nota, Duración).
- **Funciones Modulares:** Cada canción está encapsulada en su propia función (ej. `play_mario()`), lo que permite un código limpio y organizado.
- **Interfaz de Usuario (REPL):** El uso de la función `input()` permite capturar texto desde la terminal de Thonny en tiempo real, permitiendo al usuario interactuar directamente con el hardware de la Raspberry Pi Pico.

- **Codigo 1: 1_buzzer_canciones.py**

3 Actividad 2: Lectura de Señales Analógicas con Potenciómetro

3.1 Objetivo

Comprender el funcionamiento del Conversor Analógico a Digital (ADC) de la Raspberry Pi Pico. El estudiante deberá conectar un potenciómetro para variar el voltaje de entrada, leer la señal mediante MicroPython y aplicar una transformación matemática para visualizar los datos en una escala de 0 a 100 (porcentaje) en la terminal de Thonny.

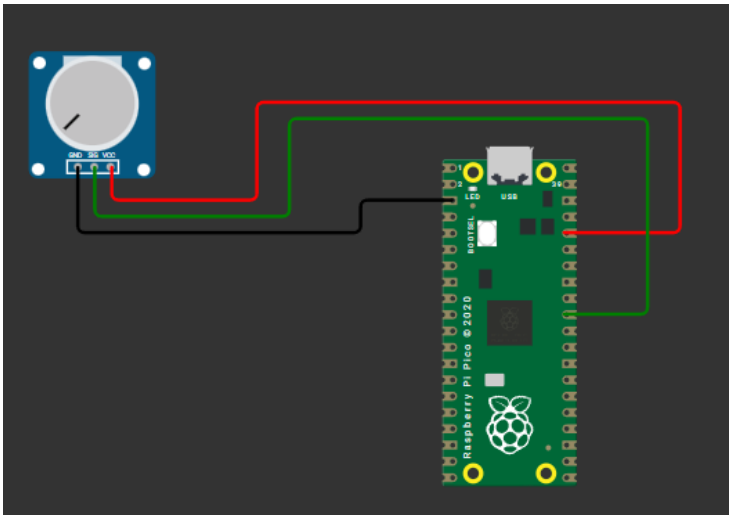

Figura 6: Esquema de conexión del potenciómetro al pin analógico GP26.

3.2 Materiales Requeridos

- 1x Raspberry Pi Pico
- 1x Protoboard
- 1x Potenciómetro (se recomienda de 10kΩ)
- Cables Jumper

3.3 Instrucciones

1. **Montaje del circuito:** Inserte el potenciómetro en la protoboard. Conecte uno de los pines de los extremos al pin de **3.3V** (Pin físico 36) y el otro extremo a **GND**. Finalmente, conecte el pin central (cursor) al pin **GP26** (Pin físico 31), el cual cuenta con soporte ADC.
2. **Carga del firmware:** Abra en Thonny el archivo de código proporcionado para la lectura del potenciómetro.
3. **Prueba de funcionamiento:** Ejecute el script, gire lentamente la perilla del potenciómetro y observe cómo varían los datos impresos en la consola inferior de Thonny. Detenga la lectura usando `Ctrl+C`.

Conceptos Clave del Código:

A diferencia de un botón que solo entrega un estado de encendido o apagado, el potenciómetro entrega un voltaje continuo que varía entre 0V y 3,3V. Para que la placa entienda esto, utilizamos las siguientes instrucciones:

- `ADC(Pin(26))`**:** Habilita el **Conversor Analógico a Digital (ADC)** en el pin GP26. Este hardware interno es el encargado de "traducir.[el] voltaje físico que entra por el cable a un número digital que MicroPython pueda procesar.
- `read_u16()`**:** Captura el nivel de voltaje actual en el pin. Como vimos anteriormente, al usar una resolución de 16 bits (`_u16`), esta función nos entregará un "valor crudo"que va desde `0` (cuando el potenciómetro entrega 0V) hasta **65535** (cuando entrega 3,3V).
- `int((valor_crudo / 65535) * 100)`**:** Es una simple regla de tres matemática. Como leer un número entre 0 y 65535 es poco intuitivo, esta fórmula divide el valor actual por el máximo posible y lo multiplica por 100, transformando la lectura en un **porcentaje (0 % a 100 %)** mucho más fácil de comprender para el usuario.

Advertencia Crítica

Verifique dos veces su cableado antes de conectar la placa al puerto USB. **Bajo ninguna circunstancia** conecte los extremos del potenciómetro al pin de 5V (VBUS). Ingresar 5V por el pin GP26 quemará permanentemente el conversor ADC de la Raspberry Pi Pico.

- **Codigo 2: 2_potenciometro.py**

4 Actividad 3: Integración de Sensores y Actuadores

4.1 Objetivo

Integrar la lectura analógica (entrada) y la señal PWM (salida) en un solo sistema interactivo. El estudiante deberá utilizar el potenciómetro para modificar en tiempo real la frecuencia de la señal enviada al buzzer, creando un control de tono dinámico.

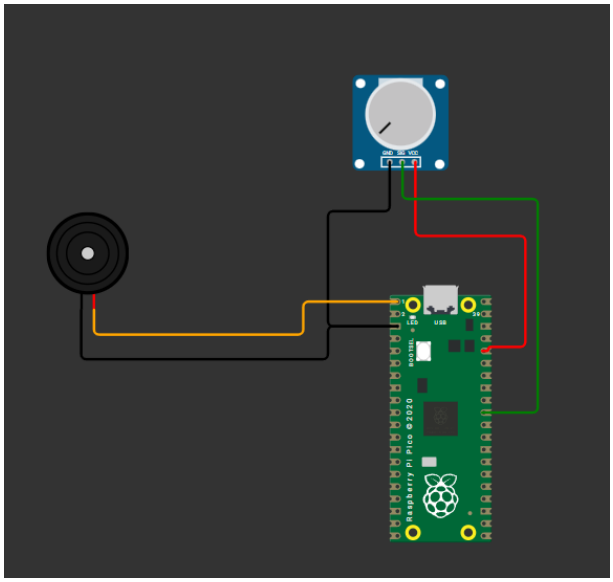

Figura 7: Esquema del circuito integrado: Potenciómetro en GP26 y Buzzer en GP0.

4.2 Materiales Requeridos

- 1x Raspberry Pi Pico
- 1x Protoboard
- 1x Potenciómetro (10kΩ)
- 1x Buzzer (Pasivo)
- Cables Jumper

4.3 Instrucciones

1. **Montaje del circuito:** Mantenga las conexiones realizadas en las Actividades 1 y 2. Asegúrese de tener el buzzer conectado al pin **GP0** y el potenciómetro al pin **GP26** (ADC0), compartiendo correctamente las líneas de alimentación (3.3V) y tierra (GND) en su protoboard.
2. **Carga del firmware:** Abra en Thonny el archivo de código correspondiente a la integración de sensor y actuador (Theremin).
3. **Prueba de funcionamiento:** Ejecute el script. Gire la perilla lentamente y escuche cómo cambia la altura (tono) del sonido emitido, mientras observa en la consola de Thonny cómo los valores se transforman en tiempo real.

Nota sobre Frecuencias de Resonancia

Notará que el buzzer suena mucho más fuerte (mayor volumen aparente) en ciertas frecuencias específicas al girar el potenciómetro, a pesar de que el ciclo de trabajo se mantiene constante al 50 %. Esto se debe a la **frecuencia de resonancia** del componente físico, el punto donde su estructura mecánica vibra con mayor eficiencia.

Conceptos Clave del Código:

El gran desafío de esta actividad es que tenemos dos componentes que "hablan" en escalas numéricas completamente distintas. El potenciómetro entrega valores de 0 a 65535, pero el buzzer solo puede reproducir bien frecuencias que van desde los 200 Hz hasta los 2000 Hz. Para solucionar esto, el código utiliza:

- **La función `mapear()`:** Es una fórmula matemática (basada en la interpolación lineal) que ustedes encontrarán definida al inicio del código. Su trabajo es "traducir"proporcionalmente un número de una escala a otra. Si el potenciómetro está exactamente a la mitad de su giro (valor $\approx$ 32768), la función calculará exactamente la mitad de la escala de frecuencias (valor $\approx$ 1100 Hz).
- **`buzzer.freq(nueva_frecuencia)`:** Dentro del ciclo principal (`while True`), el programa lee constantemente el sensor, calcula la conversión matemática y actualiza este parámetro de forma instantánea. Esto es lo que genera la sensación de control interactivo en tiempo real.

- **Codigo 3: 3_potenciometro_y_buzzer.py**
- **Codigo 3: 3_variador_de_volumen.py**

5 Referencias

1 Link 1

2 Link 2

3 Link 3